2609 CS460

Computer Vision

Homework2: Image Filtering, Processing and Feature Detection

Name: 李昌硕

Student ID: 1240000787

Overview

This report covers Tasks 1-5 of Assignment 2. The completed code files are `filters.py` (Tasks 1-3), `corners.py` (Task 4) and `blob_detection.py` (Task 5). Every script was run on a GitHub Actions Ubuntu runner (Python 3.11, NumPy, SciPy, Matplotlib, Pillow); all figures and numbers in this report come from that run. A separate verification script ran 32 automatic checks, and all 32 passed. Among them: `convolve()` matches `scipy.ndimage.convolve(mode='constant')` exactly for nine kernel shapes (including even sizes), the convolutional Harris detector matches a quadruple-loop brute-force version, and the Task 5 Gaussian blur matches SciPy's Gaussian filter to within 10^{-14}. The full check list is in the appendix.

Task 1: Convolution and Gaussian Filter

Task 1 (a) Separability of the 2D Gaussian

The 2D Gaussian factorises into a product of two 1D Gaussians:

$$G(x,y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} = \left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{y^2}{2\sigma^2}}\right)\left(\frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}\right) = G(y)\,G(x)$$

Pick a filter size k with sample positions $x_j, y_i \in \{-(k-1)/2, \ldots, (k-1)/2\}$. Define the vertical filter $G_y \in R^{k\times 1}$ with entries $G_y[i] = G(y_i)$, the horizontal filter $G_x \in R^{1\times k}$ with $G_x[j] = G(x_j)$, and the 2D filter $G \in R^{k\times k}$ with $G[i, j] = G(x_j, y_i)$. Looking at individual elements:

$$G[i,j] = G(x_j, y_i) = G(y_i)\,G(x_j) = G_y[i]\,G_x[j] = (G_y G_x)[i,j] \quad \Rightarrow \quad G = G_y G_x = G_y * G_x$$

The last equality uses the given fact that convolving a k×1 column with a 1×k row is their outer product. Because convolution is associative, for any image I

$$I * G = I * (G_y * G_x) = (I * G_y) * G_x$$

So filtering with the k×k Gaussian is the same as filtering vertically with G_y and then horizontally with G_x. This costs 2k instead of k^2 multiplications per pixel. Numerical check (k = 3, σ = 0.572): max $|G - G_y G_x| = 1.11\times10^{-16}$, and max $|I*G - (I*G_y)*G_x| = 1.14\times10^{-13}$ on grace_hopper.png (floating-point round-off only).

Task 1 (b) convolve()

`convolve()` flips the kernel in both directions as soon as it receives it, zero-pads the image by (h-1)//2 rows on top, h//2 at the bottom, (w-1)//2 columns on the left and w//2 on the right, and then accumulates kernel[i, j] × (shifted image window). This is true convolution, not cross-correlation. The output size is H×W, and the kernel centre is (h//2, w//2), the same convention as SciPy, so even-sized kernels match as well. Against scipy.ndimage.convolve(mode='constant'), the maximum absolute difference is 0 for kernels of shape 1×1, 1×3, 3×1, 3×3, 5×5, 9×9, 2×2, 4×3 and 2×5, and for the 9×9 LoG kernel on grace_hopper.png. For an asymmetric kernel the result differs from cross-correlation, which confirms that the flip is applied.

Listing 1. convolve() in filters.py

```
def convolve(image, kernel):
    image = np.asarray(image, dtype=float)
    kernel = np.asarray(kernel, dtype=float)
    if kernel.ndim == 1:
        kernel = kernel[None, :]
    # Flip the kernel so that the sliding-window sum below is a convolution.
    kernel = kernel[::-1, ::-1]

    H, W = image.shape
    h, w = kernel.shape
    # The kernel center is (h // 2, w // 2), the same convention as
    # scipy.ndimage.convolve, so even-sized kernels also match scipy.
```

```
pad_top, pad_bottom = (h - 1) // 2, h // 2
pad_left, pad_right = (w - 1) // 2, w // 2
padded = np.pad(image, ((pad_top, pad_bottom), (pad_left, pad_right)),
                mode='constant', constant_values=0)

output = np.zeros((H, W), dtype=float)
for i in range(h):
    for j in range(w):
        output += kernel[i, j] * padded[i:i + H, j:j + W]
return output
```

Task 1 (c) 3×3 Gaussian, σ = 0.572

Kernel from the 2D formula (the values sum to 1.000089):

```
[ 0.0229  0.1055  0.0229 ]
[ 0.1055  0.4864  0.1055 ]
[ 0.0229  0.1055  0.0229 ]
```

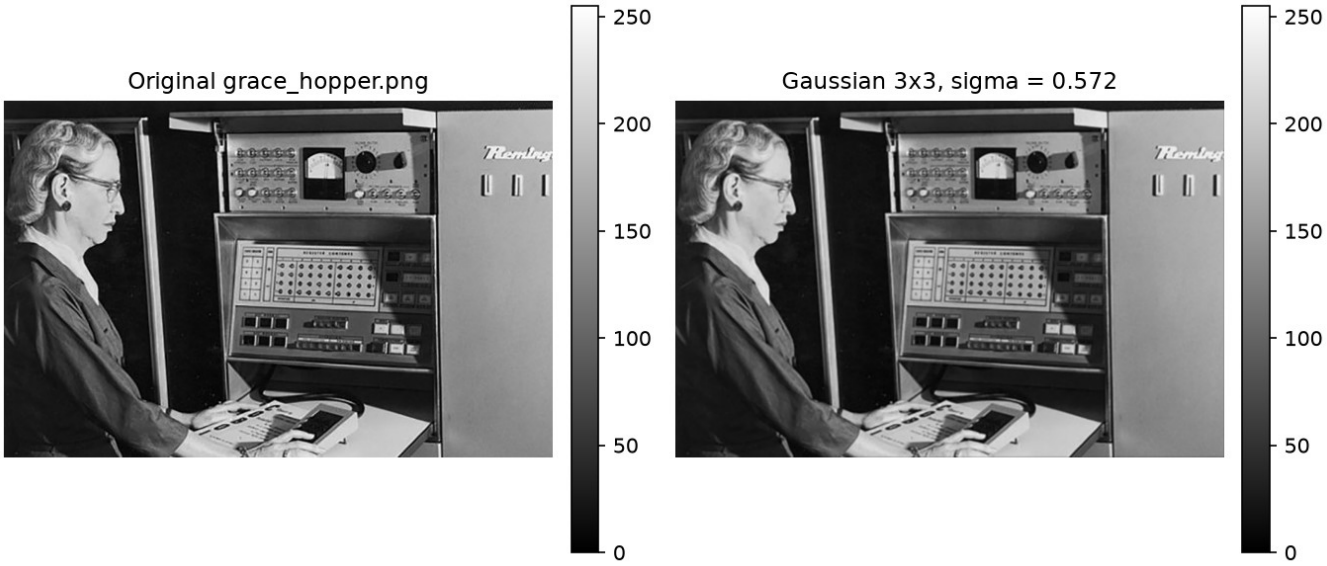


Figure 1. grace_hopper.png before and after the 3×3 Gaussian filter (σ = 0.572); both images use vmin = 0, vmax = 255.

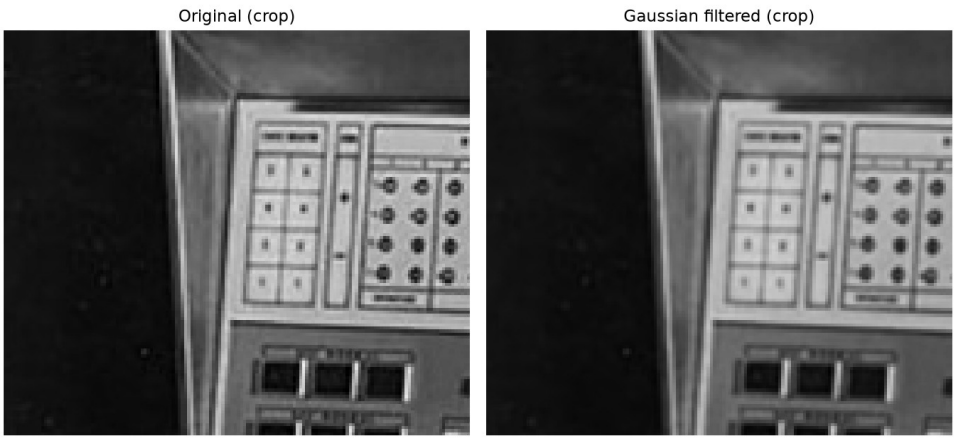


Figure 2. Close-up crop: the filtered image is slightly blurred.

Effect of Gaussian filtering: it replaces each pixel by a Gaussian-weighted average of its neighbourhood, so the image is slightly blurred: noise and fine texture are smoothed out and edges become softer, while the overall brightness is unchanged.

Task 1 (d) Why a smoothing filter should sum to 1

Filter	Kernel sum	Output min	Output max	Range	Mean
None (original)	-	0.00	255.00	255.00	95.56
3×3, σ = 0.572 (raw)	1.000089	3.06	248.35	245.29	95.46
3×3, σ = 0.572 (divided by sum)	1.000000	3.06	248.33	245.27	95.45
3×3, σ = 2 (raw, not normalised)	0.304192	1.22	74.97	73.75	29.00

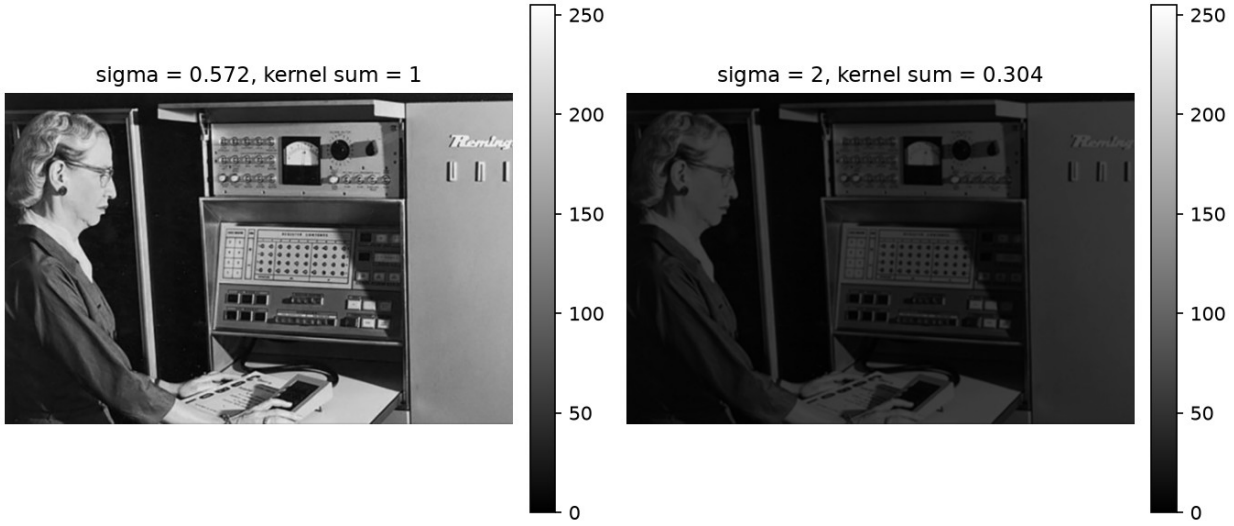


Figure 3. Left: σ = 0.572 kernel normalised to sum 1. Right: σ = 2 kernel used as is (sum 0.304). Same display range vmin = 0, vmax = 255.

With σ = 0.572 the kernel already sums to 1.000089, so the output keeps the original intensity scale (min 3.06, max 248.33, range 245.27). The minimum rises and the maximum falls only because extreme pixels are averaged with their neighbours. With σ = 2 the 3×3 window keeps only 0.304 of the Gaussian's mass (the curve is very wide, so most of its mass lies outside the window). Every output value is scaled by about that factor: the mean drops from 95.56 to 29.00 (ratio 0.303) and the maximum is only 74.97, so the image looks much darker.

Reason: on a constant region I = c the output is c·Σk. A smoothing filter should only redistribute intensity between neighbouring pixels, not create or remove it. If Σk = 1, flat regions and the mean brightness (the DC component) are preserved and only local variations are averaged out. If Σk ≠ 1, the filter also rescales the whole image: Σk < 1 darkens it, and Σk > 1 brightens it and can push values past 255. Repeated filtering compounds the error as $(\Sigma k)^n$, and results computed with different kernels would no longer share an intensity scale. Dividing the kernel by its sum fixes this.

Task 1 (e) Derivative kernels

For a 1×3 kernel with taps k(-1), k(0), k(1) (centre at index 0), convolution gives

$$(I * k_x)(x,y) = \sum_{m=-1}^{1} k_x(m)\, I(x-m,y) = k_x(-1)\, I(x+1,y) + k_x(0)\, I(x,y) + k_x(1)\, I(x-1,y)$$

Matching this to I_x = I(x+1, y) - I(x-1, y) gives $k_x(-1) = 1$, $k_x(0) = 0$, $k_x(1) = -1$. Written left to right, the kernel array is therefore [1, 0, -1]. Note that the flip in convolution reverses the intuitive [-1, 0, 1]. The same argument along the vertical axis (y increases downwards, i.e. along rows) gives:

```
(i)  kx = [ 1  0  -1 ]  ∈ R^(1×3)          (ii) ky = [ 1  0  -1 ]^T  ∈ R^(3×1)
```

Check: on a horizontal ramp I(x, y) = x, edge_detection() returns $I_x = 2$ and $I_y = 0$ at every interior pixel, and on a vertical ramp it returns $I_y = 2$. This is twice the unit slope, as expected.

Task 1 (f) edge_detection()

`edge_detection()` convolves the image with k_x and k_y using convolve() and returns I_x, I_y and the gradient magnitude $\sqrt{(I_x^2 + I_y^2)}$.

Listing 2. edge_detection() in filters.py

```
def edge_detection(image):
    # TODO: Fix kx, ky
    # Ix(x, y) = I(x + 1, y) - I(x - 1, y), Iy(x, y) = I(x, y + 1) - I(x, y - 1)
    kx = np.array([[1, 0, -1]])  # 1 x 3
    ky = np.array([[1], [0], [-1]])  # 3 x 1

    Ix = convolve(image, kx)
    Iy = convolve(image, ky)

    # TODO: Use Ix, Iy to calculate grad_magnitude
    grad_magnitude = np.sqrt(Ix ** 2 + Iy ** 2)

    return Ix, Iy, grad_magnitude
```

Task 1 (g) Gradient magnitude of the original and Gaussian-filtered images

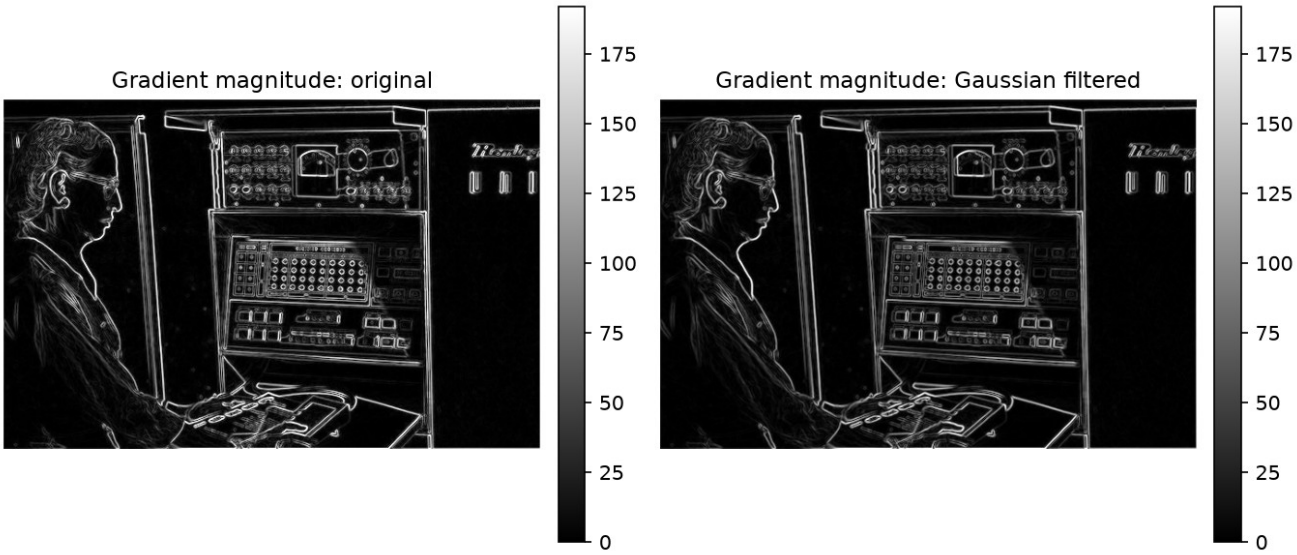


Figure 4. Gradient magnitude from edge_detection(): original image (left) and the σ = 0.572 Gaussian-filtered image (right), same display range.

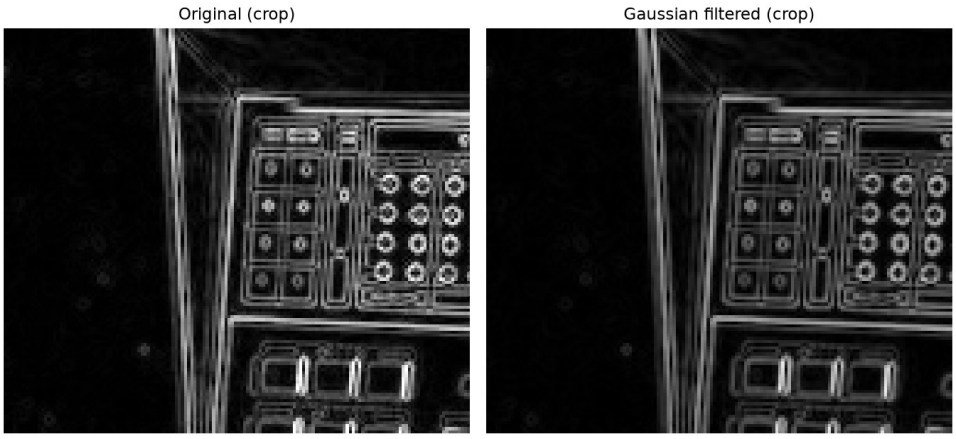


Figure 5. Close-up crop of the two gradient-magnitude images.

The gradient magnitude of the original image (mean 22.65, max 295.48) has the thinnest and sharpest edges, but also responds to film grain and fine texture, which shows up as faint speckle in flat areas such as the cabinet and wall. After Gaussian filtering (mean 19.04, max 238.98) that

speckle is weaker, and the outlines of the person, console panels, buttons and dials are smoother but slightly wider and lower in contrast. Because σ = 0.572 is small, the difference is modest, but it shows the usual trade-off: smoothing before differentiating gives up a little edge localisation for robustness to noise.

Task 2: Sobel Operator

Task 2 (a) Sobel filtering approximates the derivative of the Gaussian-filtered image

Let g = [1 2 1]. Then $G_S = g^T g = g^T * g$, and $S_x = g^T[1\ 0\ -1] = g^T * k_x$. The horizontal derivative of the Gaussian-filtered image is $(I * G_S) * k_x$. Because convolution is associative and commutative,

$$(I * G_S) * k_x = I * (G_S * k_x), \qquad G_S * k_x = g^T * (g * k_x)$$

so the image can be ignored and only the two filters $G_S * k_x$ and S_x need comparing. With $g * k_x$ = [1 2 1] ∗ [1 0 -1] = [1 2 0 -2 -1]:

```
                [ 1  2  0  -2  -1 ]              [ 1  0  -1 ]
GS * kx  =      [ 2  4  0  -4  -2 ]      Sx  =   [ 2  0  -2 ]
                [ 1  2  0  -2  -1 ]              [ 1  0  -1 ]
```

Splitting the 3×5 kernel into its centre and outer columns gives an exact identity, where d_1 = [1 0 -1] is a central difference with step 1 and d_2 = [1 0 0 0 -1] is a central difference with step 2:

$$G_S * k_x = 2\, g^T * d_1 + g^T * d_2 = 2\, S_x + g^T * d_2$$

Both terms are a vertical [1 2 1] smoothing followed by a horizontal central difference, so both measure the same horizontal derivative of the vertically smoothed image. Only the step size differs. If the image is locally linear over a few pixels, I(x+2) - I(x-2) = 2[I(x+1) - I(x-1)], so $g^T * d_2$ acts like $2S_x$. At an ideal step edge both differences equal the step height, so it acts like S_x. In both cases the extra term points the same way as S_x, and therefore

$$\frac{\partial}{\partial x}(I * G_S) \approx \tfrac{1}{2}(I * G_S) * k_x \approx c\,(I * S_x), \qquad c \in [1.5,\ 2]$$

Up to a positive constant (irrelevant for edge detection, and absorbed by normalising G_S by 1/16), the Sobel filter computes the horizontal derivative of the G_S-smoothed image. In effect it is the 3×3 truncation of the 3×5 kernel. The same argument with k_y gives S_y. On grace_hopper.png, the correlation between $I * S_x$ and $(I * G_S) * k_x$ is 0.9675. The least-squares ratio $(I * G_S) * k_x$ / $(I * S_x)$ is 2.99, i.e. essentially the step-edge value 3 (2c with c = 1.5; a smooth ramp would give 4). This shows that the strong gradients in the photograph come from sharp edges.

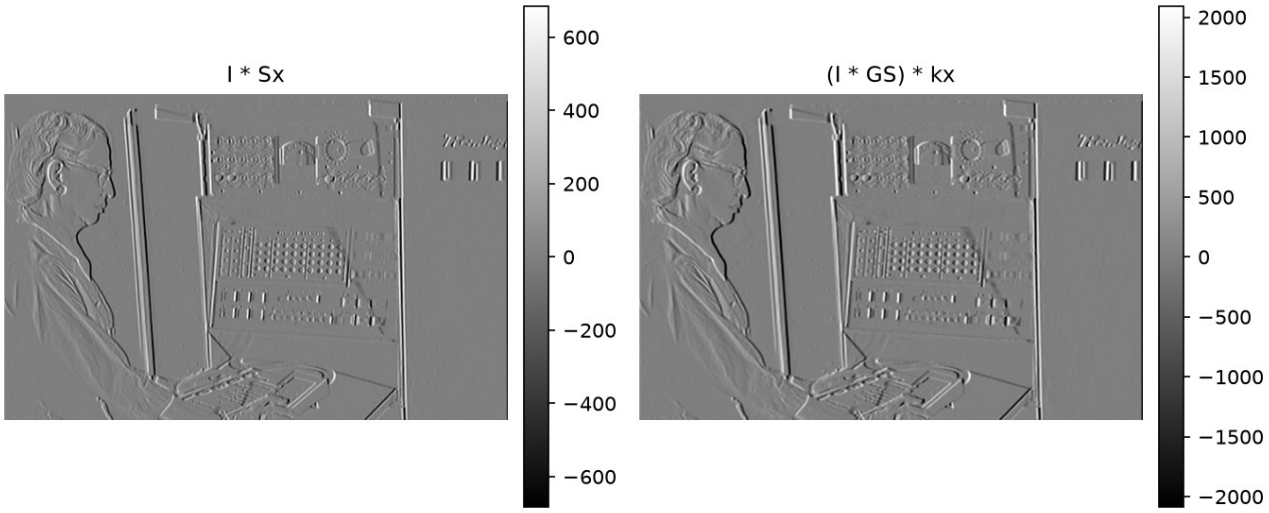


Figure 6. $I * S_x$ (left) and $(I * G_S) * k_x$ (right) are visually the same.

Task 2 (b) sobel_operator()

Listing 3. sobel_operator() in filters.py

```
def sobel_operator(image):
    # TODO: Use convolve() to complete the function
    Sx = np.array([[1, 0, -1],
                   [2, 0, -2],
                   [1, 0, -1]])
    Sy = np.array([[1, 2, 1],
                   [0, 0, 0],
                   [-1, -2, -1]])
    Gx = convolve(image, Sx)
    Gy = convolve(image, Sy)
    grad_magnitude = np.sqrt(Gx ** 2 + Gy ** 2)

    return Gx, Gy, grad_magnitude
```

Task 2 (c) Sobel outputs

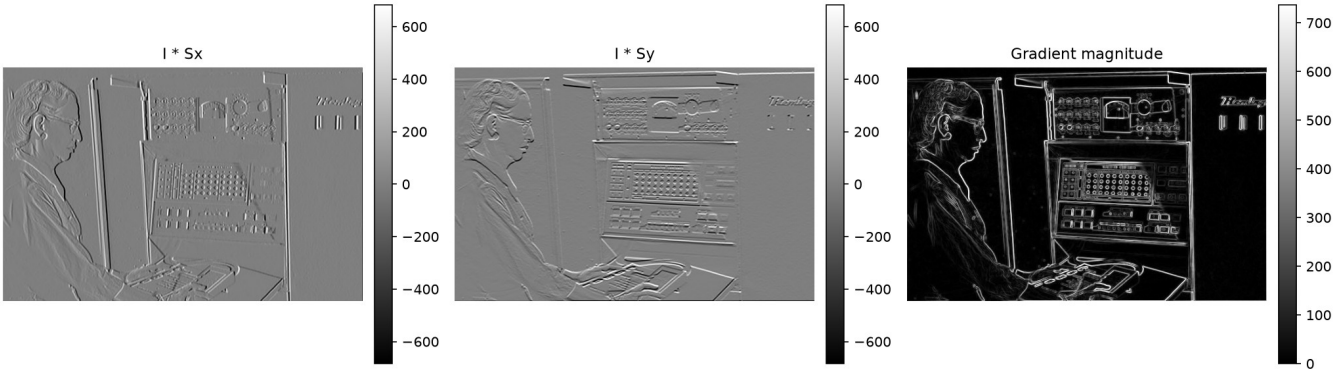


Figure 7. $I * S_x$, $I * S_y$ and the gradient magnitude $\sqrt{(G_x^2 + G_y^2)}$ for grace_hopper.png. Signed outputs use a symmetric gray scale (black negative, white positive).

$I * S_x$ responds to vertical edges (intensity changes along x) and $I * S_y$ to horizontal edges. The sign shows whether intensity increases or decreases across the edge. The magnitude combines both into an orientation-independent edge map that is smoother than the plain [1 0 -1] gradient of Task 1, because of the built-in [1 2 1] smoothing.

Task 3: LoG Filter

Task 3 (a) The two LoG filters

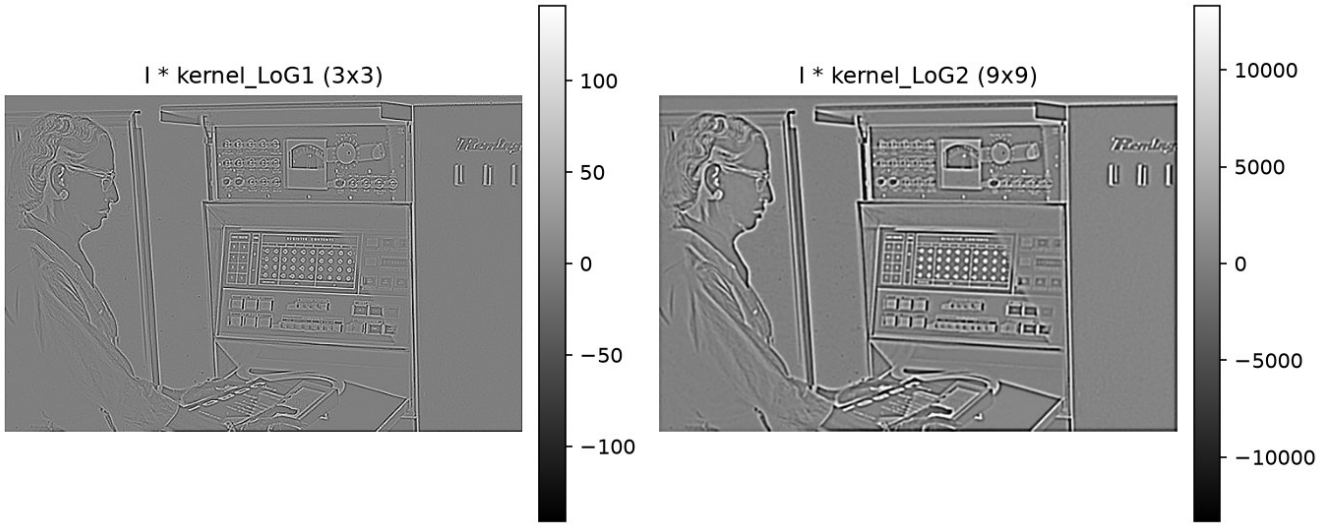


Figure 8. Signed responses to kernel_LoG1 (3×3) and kernel_LoG2 (9×9), displayed with symmetric limits (99th percentile of |response|).

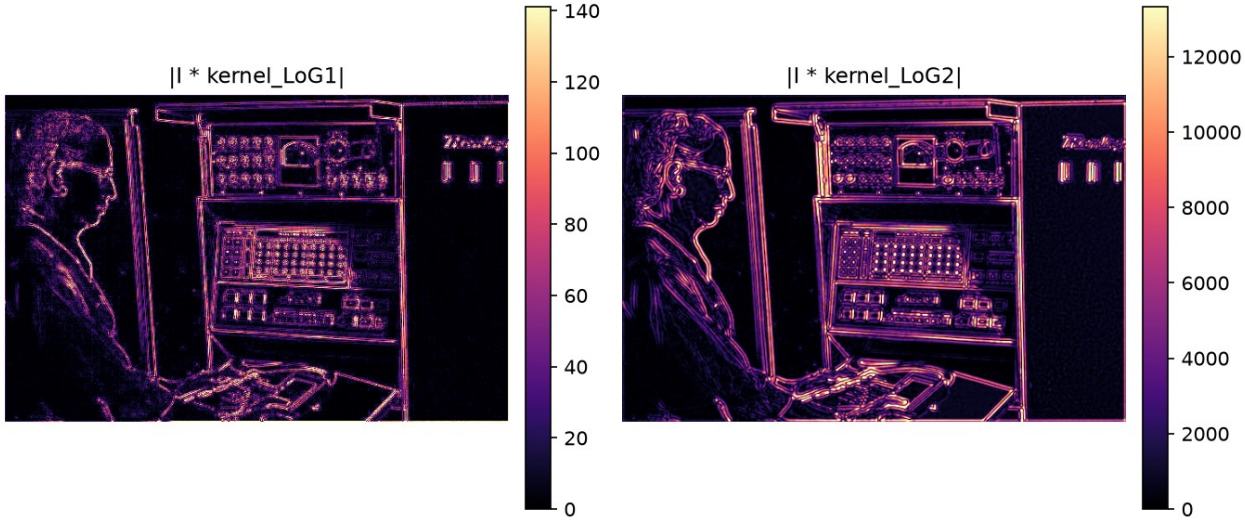


Figure 9. Absolute responses: bright = detected region (high |response|).

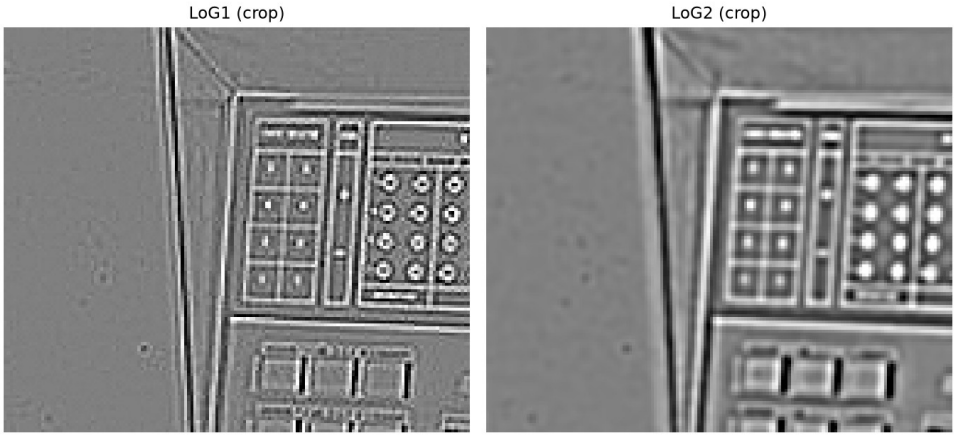


Figure 10. Close-up crop of the signed responses.

kernel_LoG1 is the 4-neighbour discrete Laplacian (sum 0). It is the LoG at the smallest possible scale, with essentially no smoothing. Its response (range -350 to 331) is sharp and noisy: it highlights single-pixel noise and grain, fine texture such as hair, and thin double lines along every edge. kernel_LoG2 (sum 4, close to 0 relative to its weights) is a sampled Laplacian of a Gaussian with σ of roughly 1.4 px: a negative centre surrounded by a positive ring. Its response

(range -29750 to 26177, larger only because its weights are larger) is much smoother. Pixel noise is averaged away, and strong responses appear as thick bands along significant edges (panel frames, the person's outline) and as blobs on structures a few pixels wide (the buttons, lamps and dials of the console, the glasses, the keyboard keys). **The difference comes from scale:** the Gaussian built into LoG2 removes detail smaller than about σ before the second derivative is taken, while LoG1 differentiates the raw pixels.

Can they detect edges? Yes, but not as maxima. At a step edge, the second derivative is strongly positive on one side and strongly negative on the other, and zero exactly on the edge. Edges are therefore the *zero crossings* between paired high-|response| bands (Marr-Hildreth). LoG2 gives cleaner zero crossings; those of LoG1 are dominated by noise. **Anything else?** LoG is the classic *blob detector*: a bright spot of radius about √2σ produces a strong negative extremum at its centre (the centre weight is negative), and a dark spot produces a positive one. LoG also responds strongly at corners, line endings, thin ridges/lines (which have high curvature in intensity), and isolated points. Flat regions and linear ramps give zero response because the kernels sum to (nearly) 0.

Task 3 (b) Approximating LoG with a Difference of Gaussians

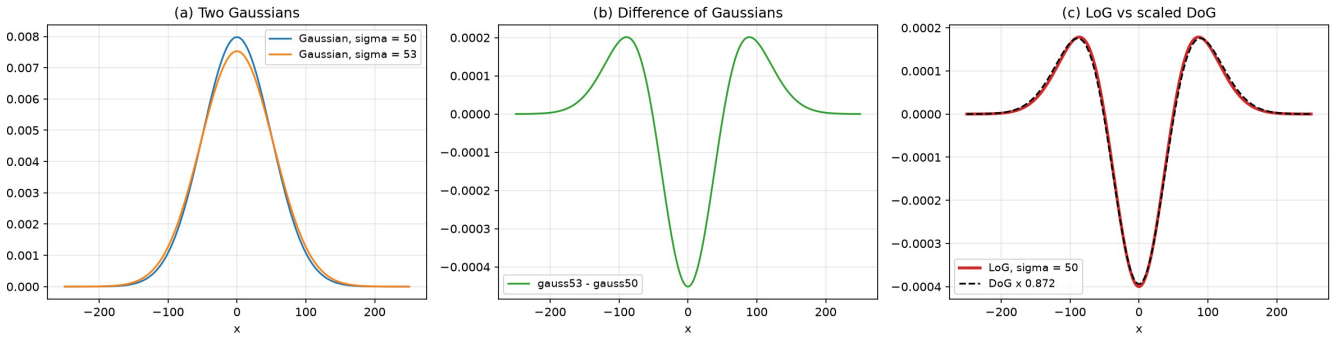


Figure 11. (a) Gaussians with σ = 50 and σ = 53 from log1d.npz. (b) Their difference gauss53 - gauss50. (c) The LoG with σ = 50 and the DoG scaled by a least-squares factor of 0.872.

Subtracting a narrow Gaussian from a slightly wider one leaves a profile with a negative centre (the narrow Gaussian is taller there), positive side lobes (the wide Gaussian is larger there), and zero total area, because both Gaussians sum to 1 (DoG sum = 2.38×10^{-16}). This is exactly the 'Mexican-hat' shape of the LoG: after a single scale factor, the DoG matches log50 with a relative L2 error of only 4.6%. The reason is the heat-diffusion equation satisfied by the Gaussian:

$$\frac{\partial G_\sigma}{\partial \sigma} = \sigma \nabla^2 G_\sigma \quad \Rightarrow \quad G_{k\sigma} - G_\sigma \approx (k\sigma - \sigma)\frac{\partial G_\sigma}{\partial \sigma} = (k-1)\,\sigma^2\,\nabla^2 G_\sigma$$

A finite difference in scale is therefore a scaled LoG. The approximation becomes exact as k → 1; here k = 53/50 = 1.06. Because convolution is linear,

$$(I * G_{k\sigma}) - (I * G_\sigma) = I * (G_{k\sigma} - G_\sigma) \approx (k-1)\,\sigma^2\,(I * \nabla^2 G_\sigma)$$

So the difference of two blurred images is (a constant times) the LoG-filtered image. The constant does not move zero crossings or extrema, so edges and blobs are found at the same places. In 2D the same identity holds because the Gaussian satisfies the 2D heat equation. In practice the blurred images are needed anyway for a scale space, and they can be produced by repeatedly applying small Gaussian kernels ($G_{\sigma 1} * G_{\sigma 2} = G_{\sqrt{(\sigma 1^2+\sigma 2^2)}}$), so the LoG at many scales comes almost for free from subtractions.

Task 4: Harris Corner Detector

Task 4 (a) Implementation

The image derivatives are $I_x = I * [1\ 0\ -1]/2$ and $I_y = I * [1\ 0\ -1]^T/2$ (scipy.ndimage.convolve, zero padding). The structure tensor M at every pixel sums the products of derivatives over a window, weighted by w (a 5×5 Gaussian, σ = 1, normalised to sum 1). Each of the three distinct entries is one convolution of a product image with w, and the three are stored as a 3-channel image (M_{11}, M_{12}, M_{22}):

$$M_{11} = w * I_x^2, \quad M_{12} = w * (I_x I_y), \quad M_{22} = w * I_y^2$$

$$R = \det M - \alpha\,(\mathrm{tr}\,M)^2 = M_{11}M_{22} - M_{12}^2 - \alpha\,(M_{11} + M_{22})^2, \qquad \alpha = 0.05$$

det and trace use the closed-form expressions for 2×2 matrices, evaluated element-wise on the three channel images, so no per-pixel loop is needed. No library corner detector is used. Following the walk-through, I first wrote a brute-force version (`harris_detector_bruteforce`, four nested loops that build M explicitly and call np.linalg.det/np.trace) and compared the two: the maximum absolute difference on a 40×40 crop is 9.31×10^{-9} for responses of order 10^6, i.e. round-off only.

Listing 4. harris_detector() in corners.py

```
def harris_detector(image, window_size=(5, 5), alpha=0.05, sigma=None):
    image = np.asarray(image, dtype=float)
    # compute the derivatives
    kx = np.array([[1, 0, -1]]) / 2.0
    ky = np.array([[1], [0], [-1]]) / 2.0
    Ix = scipy.ndimage.convolve(image, kx, mode='constant', cval=0.0)
    Iy = scipy.ndimage.convolve(image, ky, mode='constant', cval=0.0)

    Ixx = Ix * Ix
    Iyy = Iy * Iy
    Ixy = Ix * Iy

    # For each image location, construct the structure tensor and calculate
    # the Harris response
    # M = sum_w w(x, y) [[Ixx, Ixy], [Ixy, Iyy]]; each entry is a Gaussian-weighted window sum.
    weights = gaussian_window(window_size, sigma)
    M = np.stack([
        scipy.ndimage.convolve(Ixx, weights, mode='constant', cval=0.0),
        scipy.ndimage.convolve(Ixy, weights, mode='constant', cval=0.0),
        scipy.ndimage.convolve(Iyy, weights, mode='constant', cval=0.0),
    ], axis=-1)

    det_M = M[..., 0] * M[..., 2] - M[..., 1] ** 2
    trace_M = M[..., 0] + M[..., 2]
    response = det_M - alpha * trace_M ** 2

    return response
```

Task 4 (b) Harris response of grace_hopper.png

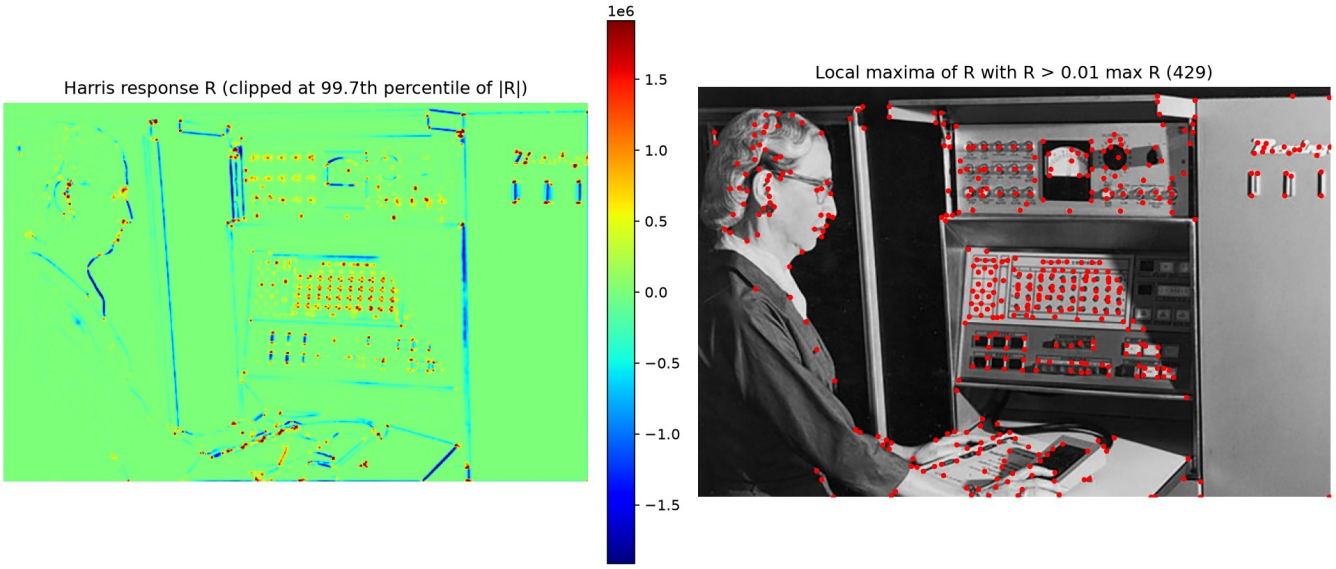


Figure 12. Left: heatmap of the Harris score R (colour limits clipped at the 99.7th percentile of |R| so that weaker structure is visible). Right: local maxima of R above 1% of max R.

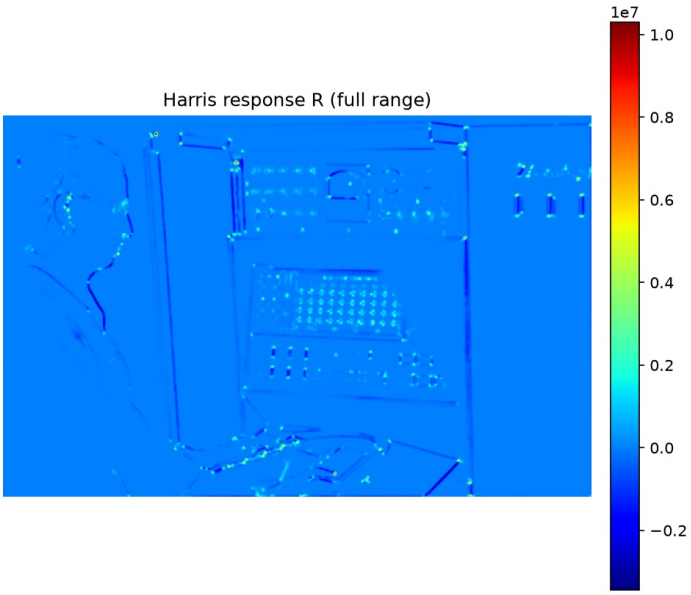


Figure 13. Heatmap of R over its full range [-3.44×10^{6}, 1.03×10^{7}].

R ranges from -3.44×10^{6} to 1.03×10^{7}. Flat regions (the wall, the cabinet surfaces, the person's clothing) have both eigenvalues small, so R ≈ 0 (green in the heatmap). Along long straight edges, such as the vertical frame of the cabinet, the panel borders and the arm outline, one eigenvalue is large and the other small, which makes R negative (blue; 29% of pixels have R < 0). The large positive peaks (red) sit where intensity changes in two directions: the regular grid of buttons and switches on the console, the corners of the panels and labels, the keyboard keys, and the glasses and curly hair. 429 local maxima exceed 1% of the maximum response.

Task 5: Blob Detection and Morphological Cell Counting

Task 5 (a) Difference-of-Gaussians blob detection

`gaussian_filter` builds a (2r+1)×(2r+1) kernel $\exp(-(x^2+y^2)/2\sigma^2)$ with r = ceil(3σ), divides it by its sum, and convolves with scipy.ndimage.convolve(mode='reflect'). No library Gaussian filter is called; the result matches scipy.ndimage.gaussian_filter to within 3×10^{-15}. `difference_of_gaussians` returns $G_{\sigma small} * I - G_{\sigma large} * I$. At the centre of a bright cell the narrow blur keeps more of the peak than the wide blur, so the response is positive there (verified on a synthetic blob). `find_blob_maxima` keeps pixels that equal the maximum of their neighbourhood_size×neighbourhood_size window and exceed threshold_rel × max(response). It then merges flat-topped plateaus (one pixel per 8-connected group) and applies greedy non-maximum suppression, dropping a weaker peak closer than neighbourhood_size/2 px to a stronger one.

Image: 002cell.png. **Scale pairs tried** (neighbourhood 9, threshold_rel 0.30): (1.5, 2.4) → 185, (2.0, 3.2) → 176, (2.5, 4.0) → 167, (3.0, 4.8) → 150, (3.5, 5.6) → 134 maxima. **Selected scales:** $\sigma_{small} = 2.5$, $\sigma_{large} = 4.0$, i.e. blob radii $\sqrt{2}\sigma \approx 3.5$-5.7 px. This matches the in-focus nuclei, whose radius is about 4-6 px, and the response is strongest near cell centres. **Response threshold:** threshold_rel = 0.3 of the maximum response 0.2178, i.e. an absolute threshold of 0.0653. **Local-maximum neighbourhood:** 9×9 maximum filter plus NMS with a 4.5 px minimum distance. **Result:** 167 detected blobs. Counting by eye on enlarged crops gives roughly 170-190 cells in this image.

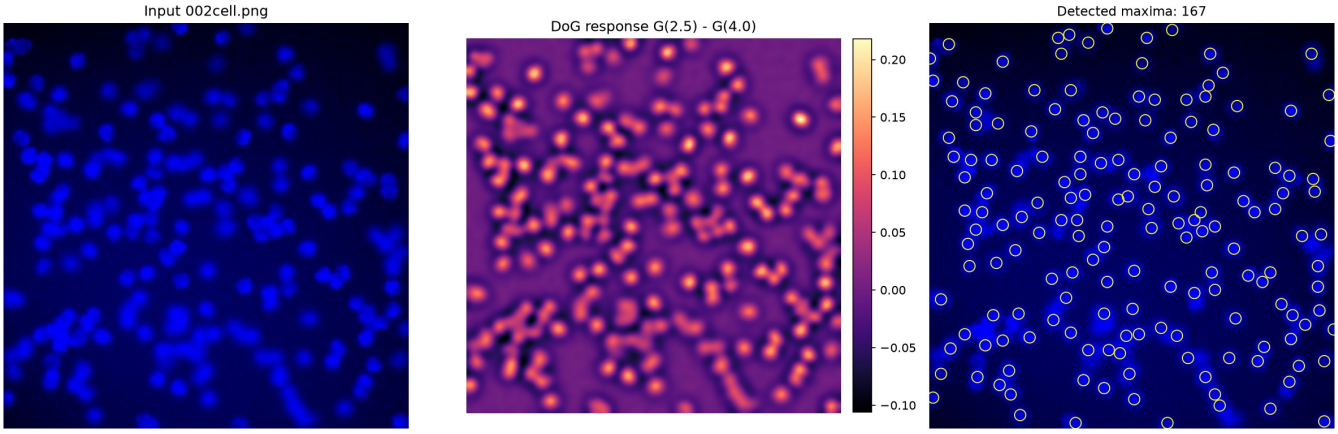


Figure 14. 002cell.png: input, DoG response (σ = 2.5, 4.0), and detected maxima (circles of radius $\sqrt{2}\sigma_{small}$).

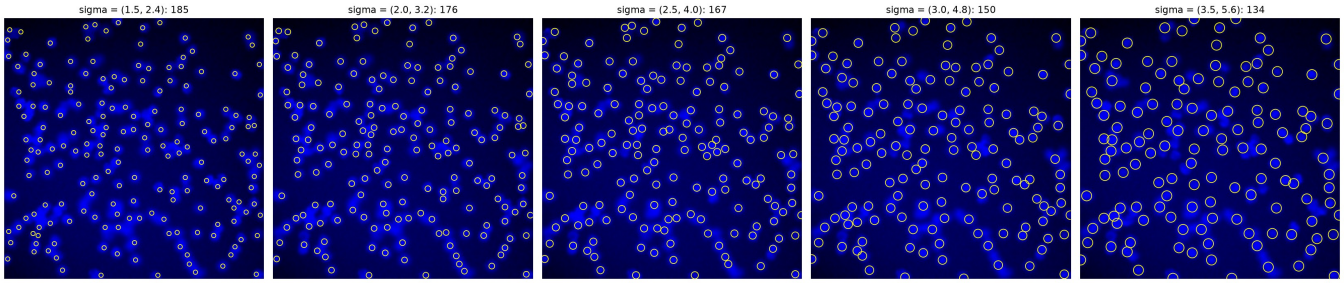


Figure 15. Detected maxima for every tried scale pair.

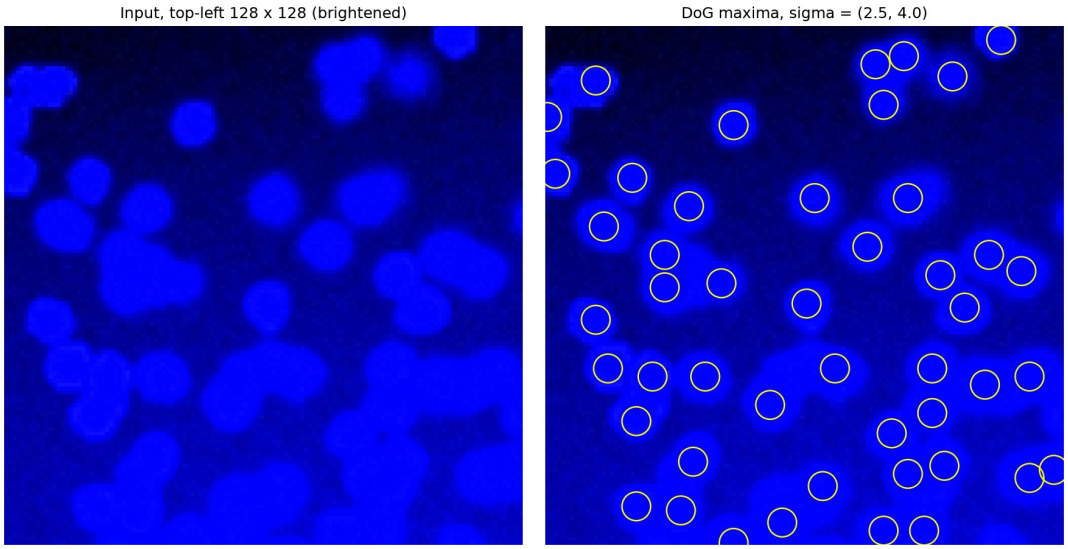


Figure 16. Enlarged top-left quarter with the selected detections.

Missed cells and false detections. Smaller scales detect more cells but start placing two maxima on large or elongated nuclei. Larger scales blur neighbouring cells into a single blob, so the count falls steadily from 185 to 134. With the selected pair, most misses occur in tight clusters and chains of overlapping cells (lower half of the image), where two or three touching nuclei produce one merged maximum or a weaker maximum within the NMS distance. Dim, out-of-focus cells on the bright hazy background in the lower part also give a weak, wide response. False or duplicate detections are rare: elongated or dumb-bell shaped cells sometimes get two maxima, and a few hits occur on bright haze between cells. The relative threshold is not the limiting factor: threshold_rel from 0.1 to 0.3 gives the same count (167), and only above 0.3 do dim cells drop out (0.4 → 149, 0.5 → 107).

Task 5 (b) Cell counting with mathematical morphology

Pipeline (morphology_mask + count_components):
1. *Scalar image:* I = max(luminance, blue) from read_gray. The microscopy images have strongly uneven illumination and a bright out-of-focus haze in parts of the field. A single global threshold therefore either merges the whole hazy region into one component or misses the darker regions. With the starter settings (threshold 0.28, no flattening), 002cell.png yields only 14 accepted components, and the largest component covers 43868 px (Figure 17). I therefore subtract a wide Gaussian blur (σ_{bg} = 10 px, made with the same self-built gaussian_filter) to remove the slowly varying background before thresholding.
2. *Threshold* the flattened image to get a binary foreground mask.
3. *Closing* (dilate, then erode; 1 iteration) fills pinholes and dark gaps inside cells, so later erosion does not break one cell into pieces.
4. *Erosion* (3 iterations) removes isolated noise pixels and cuts the narrow bridges where touching cells meet. *Dilation* (2 iterations) then grows the surviving cores back. With equal iteration counts, this erosion and dilation would form an exact opening. Using one fewer dilation keeps a one-pixel gap between cores that the erosion separated, so they are not re-merged.
5. *Structuring element:* a 3×3 cross (4-connectivity). Iterating it n times acts like a diamond of radius n, which is a cheap approximation of a disk matching the round cells.
6. *Count:* label 4-connected components with scipy.ndimage.label and keep only those with area in [15, 400] px. Below 15 px are noise fragments; above 400 px (about three or more cells) are clumps that cannot be resolved.

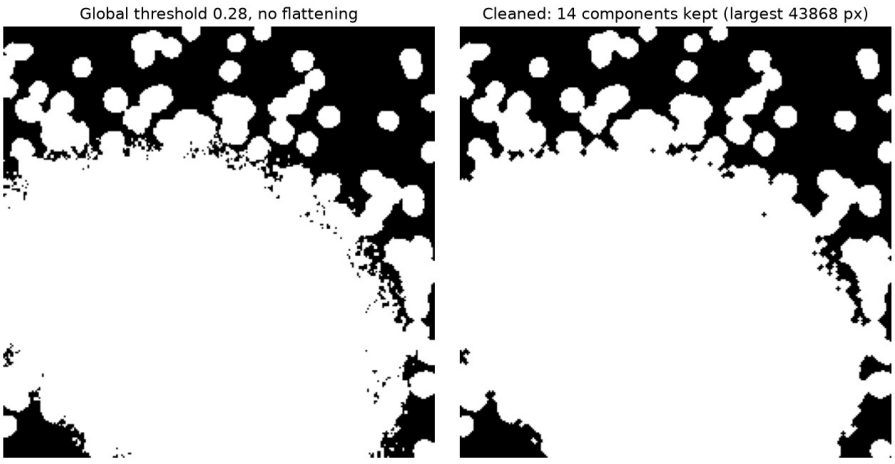


Figure 17. Baseline without background flattening (starter threshold 0.28): the hazy region becomes one giant component.

Image	Bg σ	Thresh.	Close	Erode	Dilate	Area (px)	Rejected small / large	Count
002cell.png	10	0.12	1	3	2	15-400	1 / 0	**114**
050cell.png	10	0.12	1	3	2	15-400	0 / 0	**111**
100cell.png	10	0.15	1	3	2	15-400	4 / 0	**112**
150cell.png	10	0.12	1	3	2	15-400	1 / 1	**129**

Table 1. Morphology parameters and final counts (iterations of the 3×3 cross; 001cell.png not used).

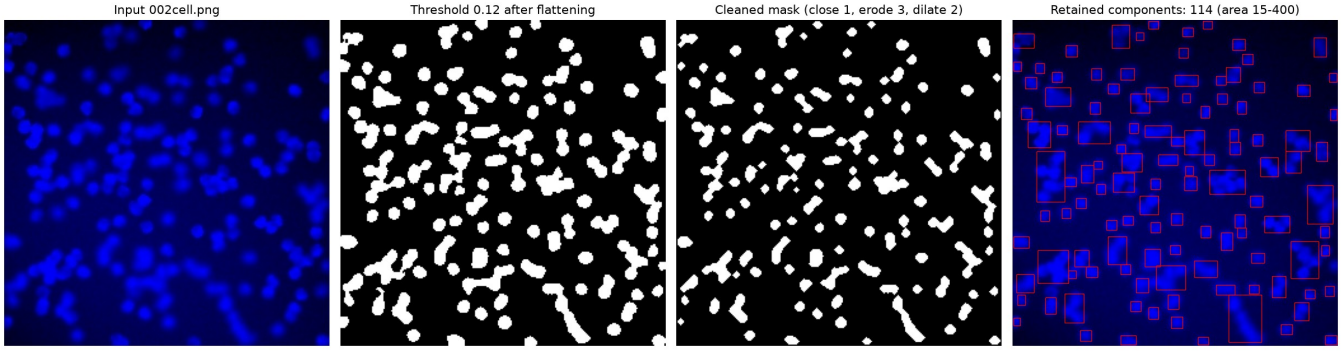


Figure 18. 002cell.png: input, threshold mask after flattening, cleaned binary mask, and retained components (red boxes): 114 cells.

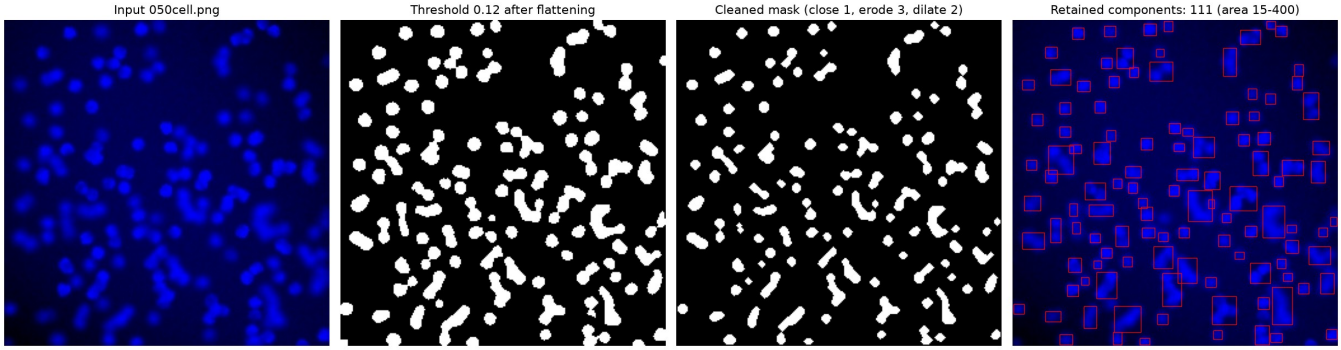


Figure 19. 050cell.png: input, threshold mask after flattening, cleaned binary mask, and retained components (red boxes): 111 cells.

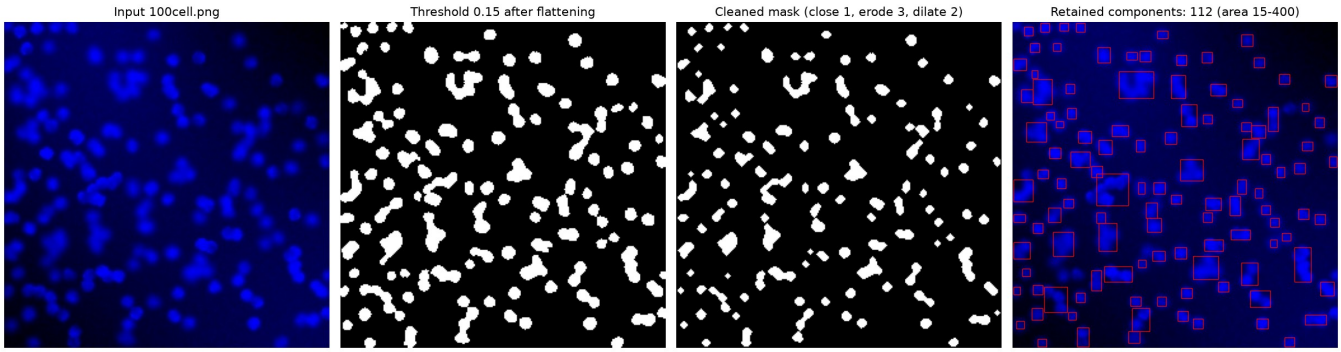


Figure 20. 100cell.png: input, threshold mask after flattening, cleaned binary mask, and retained components (red boxes): 112 cells.

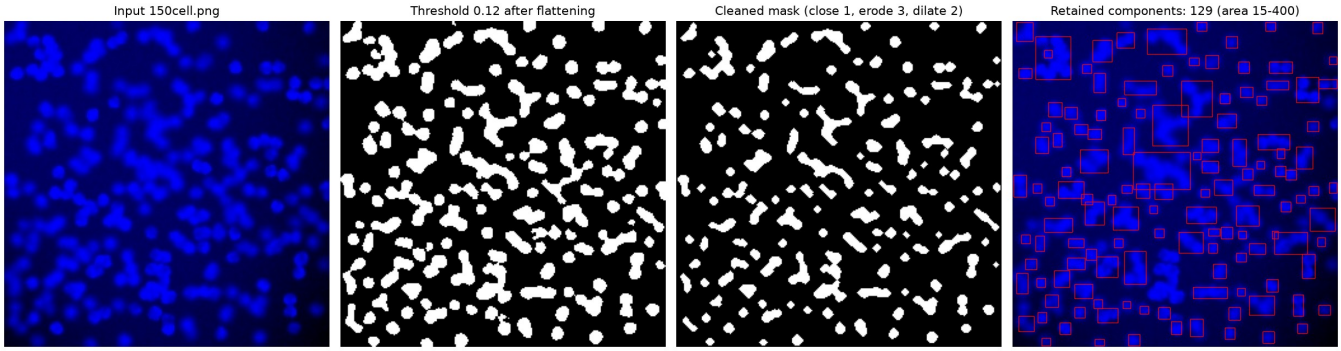


Figure 21. 150cell.png: input, threshold mask after flattening, cleaned binary mask, and retained components (red boxes): 129 cells.

The threshold was tuned per image within a small range: 0.12 for 002, 050 and 150, and 0.15 for 100cell.png, which has more haze between cells. The other settings are the same for all four images. In every image, around 0.12-0.15 the counts change by only a few cells when the threshold or the erosion/dilation counts are perturbed (see the sweeps in 5(c)).

Task 5 (c) Comparison and analysis

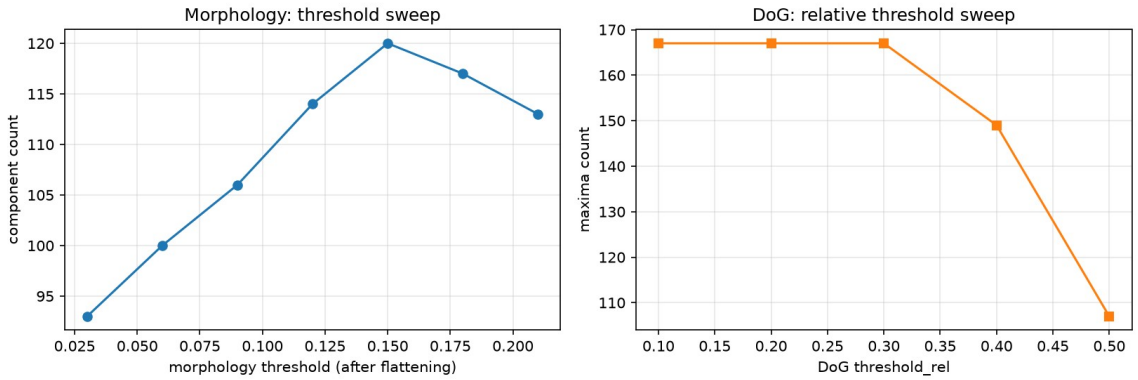


Figure 22. One-parameter sweeps on 002cell.png: morphology threshold (other settings fixed) and DoG threshold_rel.

On 002cell.png, the DoG detector finds **167** blobs and the morphology pipeline counts **114** components, against a rough visual count of 170-190. DoG works on the intensity profile directly: each nucleus is a local bump, so even touching cells keep separate maxima. Its errors are merging of very tight chains into one maximum, occasional double maxima on elongated cells, and loss of the faintest out-of-focus cells. Morphology must first commit to a binary mask, and its dominant error is *under-segmentation*: overlapping cells share one connected component, or form a clump larger than max_area that is discarded entirely. Smaller errors are dim cells lost below the threshold and, rarely, one cell split into two fragments. One parameter

change shows the trade-off clearly: raising the morphology threshold (0.03 → 93, 0.06 → 100, 0.09 → 106, 0.12 → 114, 0.15 → 120, 0.18 → 117, 0.21 → 113) increases the count, because a higher level cuts through the darker valleys between touching cells and separates them. Beyond about 0.15-0.18, dim cells shrink below min_area and disappear, so the curve flattens and then drops. Changing the erosion count (with dilation 2: 1 → 88, 2 → 97, 3 → 114, 4 → 103) behaves similarly. For DoG, raising threshold_rel from 0.3 to 0.5 removes dim cells (167 → 107). Overall, DoG maxima are better suited to dense, touching cells, while morphology gives cell shapes and areas but undercounts clusters unless a splitting step (e.g. watershed) is added.

Appendix A: Cloud verification results

Output of cloud_check/verify.py on the GitHub Actions runner (Python 3.11, Ubuntu).

Result	Check	Detail
PASS	convolve vs scipy, kernel 3x3	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 1x3	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 3x1	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 5x5	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 9x9	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 1x1	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 2x2	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 4x3	max abs diff = 0.000e+00
PASS	convolve vs scipy, kernel 2x5	max abs diff = 0.000e+00
PASS	convolve vs scipy, grace_hopper with kernel_LoG2	max abs diff = 0.000e+00
PASS	convolve is not cross-correlation (asymmetric kernel differs)	max |convolve - correlate| = 1.084e+01
PASS	filters.py does not import scipy	
PASS	2D Gaussian = Gy @ Gx	max diff = 1.110e-16
PASS	I*G = (I*Gy)*Gx on the full image	max abs diff = 1.137e-13
PASS	edge_detection on horizontal ramp	Ix = 2, Iy = 0 (interior): Ix interior unique = [2.]
PASS	edge_detection on vertical ramp	Iy = 2, Ix = 0 (interior): Iy interior unique = [2.]
PASS	GS * kx (full)	
PASS	Sx = [1,2,1]^T * [1,0,-1]	
PASS	corr(I*Sx, (I*GS)*kx) > 0.9	corr = 0.9675
PASS	harris fast vs brute force (grace_hopper crop 40x40)	max abs diff = 9.313e-09
PASS	harris fast vs brute force (random 30x30)	max abs diff = 5.588e-08
PASS	corner_score vs brute-force SSD, (u, v) = (5, 5)	max abs diff = 0.000e+00
PASS	corner_score vs brute-force SSD, (u, v) = (0, 5)	max abs diff = 0.000e+00
PASS	corner_score vs brute-force SSD, (u, v) = (-5, 0)	max abs diff = 0.000e+00
PASS	blob gaussian_filter vs scipy, sigma = 1.5	max abs diff = 9.992e-16, kernel sum = 1.000000000000
PASS	blob gaussian_filter vs scipy, sigma = 2.5	max abs diff = 1.554e-15, kernel sum = 1.000000000000
PASS	blob gaussian_filter vs scipy, sigma = 4.0	max abs diff = 2.331e-15, kernel sum = 1.000000000000
PASS	DoG positive at bright blob center	response = 0.2296, argmax = (32, 32)
PASS	one maximum on a single synthetic blob	maxima at [(32, 32)]
PASS	no scipy gaussian_filter call in blob_detection.py	
PASS	no library blob detector / cell counter	
PASS	every ndimage.convolve in blob_detection.py uses mode='reflect'	1 call(s)

Appendix B: Task 5 implementation

Listing. gaussian_filter() in blob_detection.py

```
def gaussian_filter(image, sigma):
    kernel = gaussian_kernel(sigma)
    return ndimage.convolve(np.asarray(image, dtype=np.float64), kernel, mode="reflect")
```

Listing. difference_of_gaussians() in blob_detection.py

```
def difference_of_gaussians(image, sigma_small, sigma_large):
    # At the center of a bright blob the narrow blur keeps more of the peak
    # than the wide blur, so G_small - G_large is positive there.
    return gaussian_filter(image, sigma_small) - gaussian_filter(image, sigma_large)
```

Listing. find_blob_maxima() in blob_detection.py

```
def find_blob_maxima(response, neighborhood_size=9, threshold_rel=0.30):
    response = np.asarray(response, dtype=np.float64)
    local_max = response == ndimage.maximum_filter(
        response, size=neighborhood_size, mode="reflect"
    )
    threshold = threshold_rel * response.max()
    candidates = local_max & (response > threshold) & (response > 0)

    # A flat-topped peak gives several equal maxima; keep one pixel per
    # 8-connected plateau.
    labels, n = ndimage.label(candidates, structure=np.ones((3, 3)))
    maxima = np.zeros(response.shape, dtype=bool)
    if n == 0:
        return maxima
    peaks = ndimage.maximum_position(response, labels, index=np.arange(1, n + 1))
    peaks = sorted(peaks, key=lambda p: response[p], reverse=True)

    # Greedy non-maximum suppression: a weaker peak closer than half the
    # neighborhood to a stronger one is treated as the same cell.
    min_dist = neighborhood_size / 2.0
    kept = []
    for r, c in peaks:
        if all((r - kr) ** 2 + (c - kc) ** 2 >= min_dist ** 2 for kr, kc in kept):
            kept.append((r, c))
    for r, c in kept:
        maxima[r, c] = True
    return maxima
```

Listing. morphology_mask() in blob_detection.py

```
def morphology_mask(
    image,
    threshold=0.28,
    erosion_iterations=1,
    dilation_iterations=2,
    closing_iterations=1,
    background_sigma=0,
):
    image = np.asarray(image, dtype=np.float64)
    if background_sigma > 0:
        image = image - gaussian_filter(image, background_sigma)
    mask = image > threshold
    if closing_iterations > 0:
        mask = ndimage.binary_dilation(mask, STRUCTURE, iterations=closing_iterations)
        mask = ndimage.binary_erosion(
            mask, STRUCTURE, iterations=closing_iterations, border_value=1
        )
    if erosion_iterations > 0:
        mask = ndimage.binary_erosion(mask, STRUCTURE, iterations=erosion_iterations)
    if dilation_iterations > 0:
        mask = ndimage.binary_dilation(mask, STRUCTURE, iterations=dilation_iterations)
    return mask
```

Listing. count_components() in blob_detection.py

```
def count_components(mask, min_area=12, max_area=400):
    labels, n = ndimage.label(mask, structure=STRUCTURE)
    areas = np.bincount(labels.ravel(), minlength=n + 1)[1:]
    ids = np.arange(1, n + 1)
    kept_label_ids = ids[(areas >= min_area) & (areas <= max_area)]
    return len(kept_label_ids), labels, kept_label_ids
```